Functor

Functional abstractions

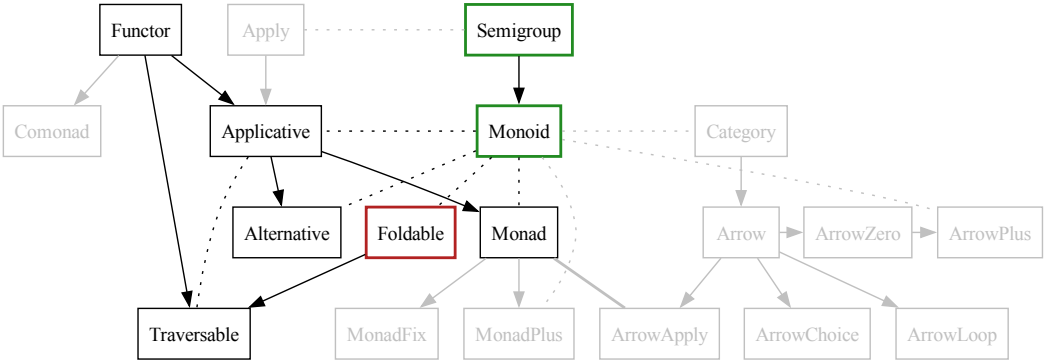
Ivan Trepakov

NSU Sys.Pro

Functional abstractions

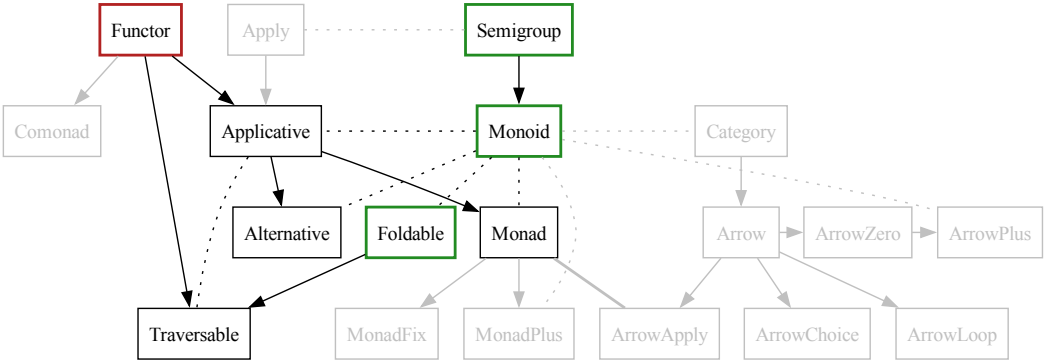
Functional abstractions

Typeclassopedia



Functional abstractions

Typeclassopedia



Functor

Data.Functor

Functor

Data.Functor

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

Functor

Data.Functor

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

Minimal complete definition

`fmap`

Functor

Data.Functor

```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

```haskell
(<$>) :: Functor f => (a -> b) -> f a -> f b
```

Minimal complete definition

`fmap`

Functor

Data.Functor

```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

```haskell
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap
```

Minimal complete definition

`fmap`

Functor

Data.Functor

```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

Minimal complete definition

`fmap`

```haskell
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap

(<&>) :: Functor f => f a -> (a -> b) -> f b
```

Functor

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

Minimal complete definition

`fmap`

Data.Functor

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap

(<&>) :: Functor f => f a -> (a -> b) -> f b
(<&>) = flip fmap
```

Functor

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
```

Minimal complete definition

`fmap`

Data.Functor

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap

(<&>) :: Functor f => f a -> (a -> b) -> f b
(<&>) = flip fmap

($>)  :: Functor f => f a -> b -> f b
($>)  = flip (<$)
```

Examples

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor


instance Functor ((,) e) where
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor


instance Functor ((,) e) where
  fmap f (x, y) = (x, f y)
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor
```

Writer monad

```haskell
instance Functor ((,) e) where
  fmap f (x, y) = (x, f y)
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor
```

Writer monad

```haskell
instance Functor ((,) e) where
  fmap f (x, y) = (x, f y)
```

```haskell
instance Functor ((->) e) where
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor
```

Writer monad

```haskell
instance Functor ((,) e) where
  fmap f (x, y) = (x, f y)
```

```haskell
instance Functor ((->) e) where
  fmap :: (a -> b) -> (e -> a) -> (e -> b)
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor
```

Writer monad

```haskell
instance Functor ((,) e) where
  fmap f (x, y) = (x, f y)
```

```haskell
instance Functor ((->) e) where
  fmap :: (a -> b) -> (e -> a) -> (e -> b)
  fmap = (.)
```

Examples

```haskell
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : fmap f xs

instance Functor Maybe where
  fmap f Nothing = Nothing
  fmap f (Just v) = Just (f v)

instance Functor (Either e) where
  fmap f (Left e)  = Left e
  fmap f (Right x) = Right (f x)
```

```haskell
{-# LANGUAGE DeriveFunctor #-}
data Tree a = Empty
            | Leaf a
            | Branch (Tree a) a (Tree a)
  deriving Functor
```

Writer monad

```haskell
instance Functor ((,) e) where
  fmap f (x, y) = (x, f y)
```

Reader monad

```haskell
instance Functor ((->) e) where
  fmap :: (a -> b) -> (e -> a) -> (e -> b)
  fmap = (.)
```

Contravariant

Can this be made into `Functor`?

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)
```

Contravariant

Can this be made into Functor?

```haskell
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
```

Contravariant

Can this be made into Functor?

```haskell
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Contravariant

Can this be made into Functor?

```haskell
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Data.Functor.Contravariant

```haskell
class Contravariant f where
  contramap :: (b -> a) -> f a -> f b
  (>$) :: b -> f b -> f a
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Data.Functor.Contravariant

```
class Contravariant f where
  contramap :: (b -> a) -> f a -> f b
  (>$) :: b -> f b -> f a
```

Minimal complete definition

`contramap`

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Data.Functor.Contravariant

```
class Contravariant f where
  contramap :: (b -> a) -> f a -> f b
  (>$) :: b -> f b -> f a
```

Minimal complete definition

contramap

```
instance Contravariant (B r) where
```

Contravariant

Can this be made into Functor?

```
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Data.Functor.Contravariant

```
class Contravariant f where
  contramap :: (b -> a) -> f a -> f b
  (>$) :: b -> f b -> f a
```

Minimal complete definition

`contramap`

```
instance Contravariant (B r) where
  contramap :: (b -> a) -> B r a -> B r b
```

Contravariant

Can this be made into Functor?

```haskell
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Data.Functor.Contravariant

```haskell
class Contravariant f where
  contramap :: (b -> a) -> f a -> f b
  (>$) :: b -> f b -> f a
```

Minimal complete definition

contramap

```haskell
instance Contravariant (B r) where
  contramap :: (b -> a) -> B r a -> B r b
  contramap f (B g) = B (g . f)
```

Contravariant

Can this be made into Functor?

```haskell
newtype B r a = B (a -> r)

instance Functor (B r) where
  fmap :: (a -> b) -> B r a -> B r b
  fmap f (B g) = B (_)

-- Have: f  :: a -> b
--       g  :: a -> r
-- Want: _  :: b -> r
-- Need: f' :: b -> a
```

Can’t be Functor!

Data.Functor.Contravariant

```haskell
class Contravariant f where
  contramap :: (b -> a) -> f a -> f b
  (>$) :: b -> f b -> f a
```

Minimal complete definition

contramap

```haskell
instance Contravariant (B r) where
  contramap :: (b -> a) -> B r a -> B r b
  contramap f (B g) = B (g . f)
```

Can be Contravariant!

Continuation

Can this be made into `Functor`?

Continuation

Can this be made into `Functor`?

```
newtype C r a = C ((a -> r) -> r)
```

Continuation

Can this be made into `Functor`?

```
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
```

Continuation

Can this be made into Functor?

```
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
```

Continuation

Can this be made into Functor?

```
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
 fmap f (C g) = C (\h -> _)
```

Continuation

Can this be made into Functor?

```
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
 fmap f (C g) = C (\h -> _)

-- Have: f  :: a -> b
--       g  :: (a -> r) -> r
--       h  :: b -> r
```

Continuation

Can this be made into Functor?

```
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
 fmap f (C g) = C (\h -> _)

-- Have: f  :: a -> b
--       g  :: (a -> r) -> r
--       h  :: b -> r
-- Want: _  :: a -> r
```

Continuation

Can this be made into Functor?

```haskell
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
 fmap f (C g) = C (\h -> g (h . f))

-- Have: f  :: a -> b
--       g  :: (a -> r) -> r
--       h  :: b -> r
-- Want: _  :: a -> r
```

Continuation

Can this be made into Functor?

```haskell
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
 fmap f (C g) = C (\h -> g (h . f))

-- Have: f  :: a -> b
--       g  :: (a -> r) -> r
--       h  :: b -> r
-- Want: _  :: a -> r
```

Can be Functor!

Continuation

Can this be made into Functor?

```
newtype C r a = C ((a -> r) -> r)

instance Functor (C r) where
 fmap :: (a -> b) -> C r a -> C r b
 fmap f (C g) = C (\h -> g (h . f))

-- Have: f  :: a -> b
--       g  :: (a -> r) -> r
--       h  :: b -> r
-- Want: _  :: a -> r
```

Can be Functor!

Control.Monad.Cont

https://hackage.haskell.org/package/mtl/docs/Control-Monad-Cont.html#t:Cont

Laws

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Parametricity

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Parametricity

Consider generic function

```
f :: [a] -> [a]
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Parametricity

Consider generic function

```
f :: [a] -> [a]
```

Representatives

```
id        :: [a] -> [a]
const []  :: [a] -> [a]
drop 2    :: [a] -> [a]
take 2    :: [a] -> [a]
reverse   :: [a] -> [a]
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Free theorem

Parametricity

Consider generic function

```
f :: [a] -> [a]
```

Representatives

```
id        :: [a] -> [a]
const []  :: [a] -> [a]
drop 2    :: [a] -> [a]
take 2    :: [a] -> [a]
reverse   :: [a] -> [a]
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

Parametricity

Consider generic function

```
f :: [a] -> [a]
```

Representatives

```
id        :: [a] -> [a]
const []  :: [a] -> [a]
drop 2    :: [a] -> [a]
take 2    :: [a] -> [a]
reverse   :: [a] -> [a]
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

Parametricity

Consider generic function

```
f :: [a] -> [a]
```

Representatives

```
id        :: [a] -> [a]
const []  :: [a] -> [a]
drop 2    :: [a] -> [a]
take 2    :: [a] -> [a]
reverse   :: [a] -> [a]
```

Example

```
f = reverse
map g . reverse = reverse . map g
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Parametricity

Consider generic function

```
f :: [a] -> [a]
```

Representatives

```
id        :: [a] -> [a]
const []  :: [a] -> [a]
drop 2    :: [a] -> [a]
take 2    :: [a] -> [a]
reverse   :: [a] -> [a]
```

Example

```
f = reverse
map g . reverse = reverse . map g
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Proof: (2) can be derived from (1)

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Proof: (2) can be derived from (1)

Suppose law (1) holds for fmap:

```
(1) fmap id = id
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Proof: (2) can be derived from (1)

Suppose law (1) holds for fmap:

```
(1) fmap id = id
```

Then using (free) and

```
fmap' = fmap, a = f,  b = g
c = id, d = (f . g)
```

We get

```
  fmap f . fmap g
= fmap id . fmap (f .g) -- (free)
= id . fmap (f . g)     -- (1)
= fmap (f . g)
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Proof: (2) can be derived from (1)

Suppose law (1) holds for fmap:

```
(1) fmap id = id
```

Then using (free) and

```
fmap' = fmap, a = f,  b = g
c = id, d = (f . g)
```

We get

```
  fmap f . fmap g
= fmap id . fmap (f .g) -- (free)
= id . fmap (f . g)     -- (1)
= fmap (f . g)
```

```
(2) fmap f . fmap g = fmap (f . g)
```

□

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Uniqueness

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]

forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]

forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Uniqueness

If `fmap` satisfies (1) (and (2)) then it is unique

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]

forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]

forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Uniqueness

If `fmap` satisfies (1) (and (2)) then it is unique

Each function

`fmap' :: (a -> b) -> f a -> f b`

satisfying (1) (and (2)) is equivalent to `fmap`:

`fmap' ~ fmap`

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Uniqueness

If fmap satisfies (1) (and (2)) then it is unique

Each function

`fmap' :: (a -> b) -> f a -> f b`

satisfying (1) (and (2)) is equivalent to fmap:

`fmap' ~ fmap`

Proof

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Uniqueness

If `fmap` satisfies (1) (and (2)) then it is unique

Each function

`fmap' :: (a -> b) -> f a -> f b`

satisfying (1) (and (2)) is equivalent to `fmap`:

`fmap' ~ fmap`

Proof

Suppose there is `fmap' != fmap` that satisfies (1)

```
  fmap' id
= id          -- (1) fmap'
= fmap id     -- (1) fmap
```

□

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell `(2)` can be derived from `(1)` due to parametricity!

Free theorem

```
f :: [a] -> [a]

forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]

forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Examples of unlawful functors

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Examples of unlawful functors

Breaks both laws

```
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : f x
                        : fmap f xs
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]
```

```
forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]
```

```
forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Examples of unlawful functors

Breaks both laws

```
instance Functor [] where
  fmap f [] = []
  fmap f (x : xs) = f x : f x
                        : fmap f xs
```

Breaks (1), but not (2)

```
instance Functor Maybe where
  fmap f _ = Nothing
```

Laws

Functor laws

```
(1) fmap id = id
(2) fmap f . fmap g = fmap (f . g)
```

In Haskell (2) can be derived from (1) due to parametricity!

Free theorem

```
f :: [a] -> [a]

forall g :: a -> b
  map g . f = f . map g
```

```
(free)    fmap :: (a -> b) -> [a] -> [b]

forall fmap' :: (a -> b) -> t a -> t b
  if   a . b = c . d
  then fmap a . fmap' b = fmap' c . fmap d
```

Examples of unlawful functors

Breaks (2), but not (1)

We need “bottom” (`undefined`, `error` etc)

https://github.com/quchen/articles/blob/master/second_functor_law.md#bottom-ruins-the-party

Resources

https://wiki.haskell.org/index.php?title=Typeclassopedia#Functor

https://en.wikibooks.org/wiki/Haskell/Category_theory

Q&A